

Dışbükey Olmayan Optimizasyonda Stokastik Gradyan İniş

Deterministik (GD) ve Stokastik (SGD) Optimizasyon Algoritmalarının Non-Convex
(Dışbükey Olmayan) Yüzeylerdeki Davranış Analizi

İlayda Çıtak – 25531009

Gradyan İniş

$$x_{t+1} = x_t - \eta \nabla f(x_t)$$

x_t : t'inci iterasyondaki nokta

η : öğrenme oranı / adımı (learning rate)

$\nabla f(x_t)$: fonksiyonun türevi (veya çok boyutluysa gradient)

Deterministiktir; her adım tam gradyan kullanılarak yapılır.

Fonksiyonun türevi, bulunduğu noktada hangi yöne giderken değerinin arttığını söyler. Buna bağlı olarak tam tersine gitmek istenir. Çünkü amaç değeri azaltmaktır. Bu yüzden her adım, türevin ters yönünde yapılır.

Öğrenme oranı ise dengeleyici olarak eğimin hızını kontrol eder, sistemin sağa sola zıplamasını engeller.

Stokastik Gradyan İniş

$$x_{t+1} = x_t - \eta (\nabla f(x_t) + \xi_t)$$

ξ_t : rastgelelik, yani gradyana eklenen gürültü.

Stokastiktir, yüzeyin tüm ayrıntılarını görmek yerine “sezgisel arama” davranışı sergiler.

Bu rastgelelik yüzeyin mikro detaylarına çok odaklanmadan daha geniş bir arama davranışı yaratır.

Ne zaman tercih edilir?

- Veri setleri büyük olduğunda,
- Non-convex yüzeylerde global çözüm ararken,
- Lokal minimumlardan kaçmak isteniyorsa.

Deterministikten Stokastiğe...

Amaç : ZorYüzeylerde İyi Çözüm Arayışı

Çok çukurlu bir hedef fonksiyon üzerinde deterministik gradient descent ile gelişmiş stokastik gradient descent yöntemlerini karşılaştırmak, algoritmaların yüzey üzerinde nasıl davrandığını incelemek ve *temperature-based noise*, *learning-rate decay* ve *best-loss sabitli patience* gibi modern optimizasyon bileşenlerinin etkisini deneysel olarak göstermek.

Analitik Problem Tanımı ve Gerçek Doğru

Algoritmaların başarısını ölçebilmek için önce problemin "teorik çözümü" (Ground Truth) bulunur.

- i. Fonksiyon Seçimi: $x^4 - 4x^2 + 0.5x + 6$ veri biliminde optimizasyon zorluklarını simüle etmek için kullanılan klasik bir "çift kuyulu" (double-well) yapıdır. Bir derin (Global Min) ve bir sıg (Local Min) çukur içerir.
- ii. Türev: Manuel değil SymPy-> sp.diff ile alınır.
- iii. Hessian Kontrolü: Türevin sıfır olduğu her nokta minimum değildir (maksimum veya eyer noktası olabilir). Burada hess_func(val) > 0 kontrolü ile sadece konveks (çukur) noktalar, yani gerçek minimumlar filtrelenir.

```
# =====  
# 1. ANALİTİK PROBLEM ÇÖZÜMÜ  
# =====  
x_sym = sp.symbols('x')  
  
# --- Fonksiyon tanımı ---  
f_expr = x_sym**4 - 4*x_sym**2 + 0.5*x_sym + 6 # Double-well potansiyel  
  
# --- Otomatik türevler ---  
grad_expr = sp.diff(f_expr, x_sym)  
hess_expr = sp.diff(grad_expr, x_sym)  
  
# --- Nümerik fonksiyonlar ---  
f_func = sp.lambdify(x_sym, f_expr, 'numpy')  
grad_func = sp.lambdify(x_sym, grad_expr, 'numpy')  
hess_func = sp.lambdify(x_sym, hess_expr, 'numpy')  
  
print("Sistem analiz ediliyor...")  
  
# --- Analitik kritik noktalar ---  
global_min = None  
local_mins = []  
all_critical_x = []  
  
try:  
    # --- Kökleri bul ---  
    p = sp.Poly(grad_expr, x_sym)  
    coeffs = [float(c) for c in p.all_coeffs()]  
    roots = np.roots(coeffs)  
  
    minima = []  
    for r in roots:  
        if np.isreal(r):  
            val = float(np.real(r))  
            all_critical_x.append(val)  
  
            # Minimum mu? Hessian > 0 ise  
            if hess_func(val) > 0:  
                minima.append((val, f_func(val)))  
  
    # --- Minimumları sırala ---  
    minima.sort(key=lambda p: p[1])  
    global_min = minima[0]  
    local_mins = minima[1:]  
  
    print(f"-> Analitik Global Min: x = {global_min[0]:.4f}")
```


Deterministik Yaklaşım

Amaç sadece eğim bilgisiyle nasıl davrandığını gözlemlemektir.

Ayrıca Standart GD'nin zayıf yönleri burada iki kritik müdahale ile güçlendirilmiştir:

- i. Gradient Clipping (np.clip): Polinom fonksiyonlarda türev inanılmaz büyüyebilir (Exploding Gradients). Bu durum algoritmanın kararsızlaşmasına neden olur. -50 ile 50 arasına sabitleyerek stabilize edildi.
- ii. Azalan Adım (Learning Rate Decay : $lr *= lr_decay$): Başlangıçta büyük adımlar atarken, giderek azalır. Böylece hedefin etrafında salınım yapması engellenir.

Bu işleyiş deterministiktir. Başladığı yere en yakın çukura düşer. Eğer "Local Min" tarafında başlarsa, asla "Global Min"e geçemez (Tepeyi aşacak enerjisi yoktur).

```
# =====  
# 2. GELİŞTİRİLMİŞ BATCH GD  
# =====  
def smart_batch_gd(start_x, initial_lr, lr_decay, max_steps, tolerance):  
    """Deterministik batch gradient descent"""  
    x = start_x  
    lr = initial_lr  
    path = [x]  
  
    for t in range(max_steps):  
        # --- Gradient hesapla ---  
        grad = grad_func(x)  
        grad = np.clip(grad, -50, 50) # Çok büyük gradyanları kırıp  
  
        # --- Parametre güncelle ---  
        x_new = x - lr * grad  
        x_new = np.clip(x_new, X_LIMIT_MIN, X_LIMIT_MAX)  
        path.append(x_new)  
  
        # --- Durdurma kriteri ---  
        if abs(x_new - x) < tolerance:  
            break  
  
        x = x_new  
        lr *= lr_decay # Non-konveks için LR decay  
  
    return np.array(path), x
```


Stokastik Yaklaşım

Amaç lokal minimum tuzağına düşse bile oradan çıkabilecek enerjiyi sisteme vermektir.

Kullanılan Yöntem (Simulated Annealing): Sigma (gürültü) parametresi bir "sıcaklık" (temperature) gibi davranır.

- i. Exploration (Keşif): Başlangıçta sigma yüksektir. Algoritma eğimi (gradient) dinlemez, rastgele sağa sola savrulur. Bu sayede "tepeyi" (bariyeri) aşabilir.
- ii. Exploitation (Sömürü): Zamanla sigma $\ast$ = noise_decay ile gürültü azalır. Sistem soğur ve artık eğimi takip ederek en dip noktaya (Global Min) yerleşir.

Akıllı Erken Durdurma (Early Stopping) : Eğer algoritma limiti boyunca daha iyi bir loss değeri bulamazsa, kaynak tüketmemek için döngüyü erken kırar.

```
# =====  
# 3. GELİŞTİRİLMİŞ STOCHASTIC GD (Temperature-based noise + LR decay + Best-Loss)  
# =====  
def smart_sgd_annealing(start_x, initial_lr, lr_decay,  
                        initial_noise, noise_decay,  
                        max_steps, patience_limit=20,  
                        min_loss_improve=1e-5):  
  
    x = start_x  
    lr = initial_lr  
    sigma = initial_noise  
  
    best_x = x  
    best_loss = f_func(x)  
    patience = 0  
    path = [x]  
  
    for t in range(max_steps):  
        # --- Gradient ---  
        grad = grad_func(x)  
        grad = np.clip(grad, -50, 50)  
  
        # --- Temperature-based noise ---  
        noise = np.random.normal(0, sigma)  
  
        # --- Parametre güncelle ---  
        x = x - lr * (grad + noise)  
        x = np.clip(x, X_LIMIT_MIN, X_LIMIT_MAX)  
        path.append(x)  
  
        # --- Loss hesapla ---  
        loss = f_func(x)  
  
        # --- Best-Loss tabanlı patience ---  
        if loss < best_loss - min_loss_improve:  
            best_loss = loss  
            best_x = x  
            patience = 0 # Yeni en iyi bulundu, patience sıfırlanır  
        else:  
            patience += 1 # İyileşme yoksa patience artır  
  
        # --- Early stopping ---  
        if patience >= patience_limit:  
            break  
  
        # --- LR ve Noise annealing ---  
        lr *= lr_decay  
        sigma *= noise_decay  
  
    return np.array(path), best_x
```


Başlangıç Noktası ve Hiperparametre Seçimi

Dinamik Başlangıç Noktası : Özellikle tuzağa düşmesi için local minimum ile sağ bariyer arası seçim yapılmıştır.

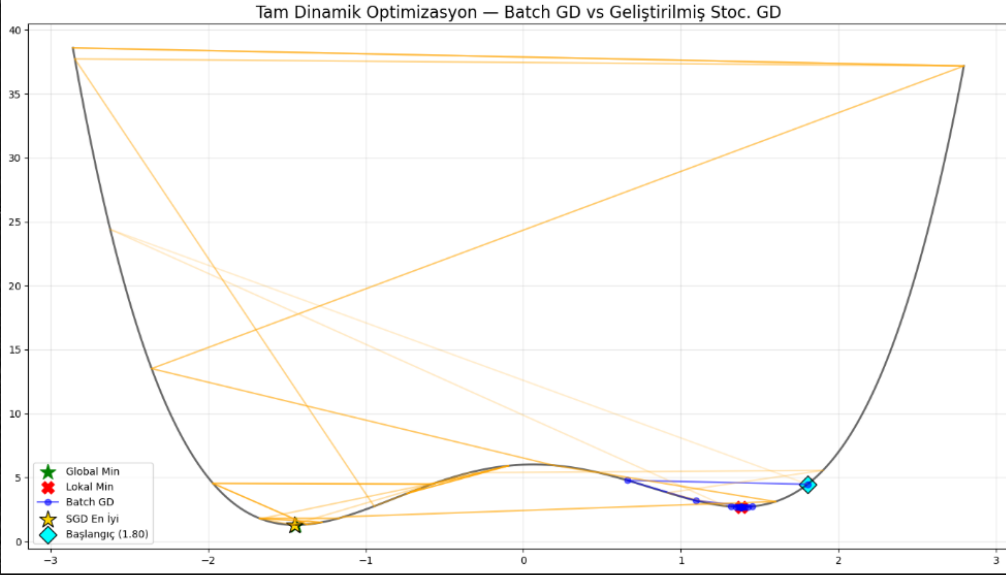
Hiperparametre Seçimi:

- i. Başlangıç Adımı(initial_lr=0.12): Agresif bir öğrenme oranıdır. Fonksiyon dik olduğu için hızlı inmek istenir.
- ii. Başlangıç Gürültüsü (initial_noise=18.0): Bu çok yüksek bir değerdir. Çünkü Lokal Min ile Global Min arasındaki tepe noktası (bariyer) yüksek. Bu bariyeri aşmak için "yüksek enerjiye" (büyük gürültüye) ihtiyaç vardır.
- iii. Azalan Gürültü(noise_decay=0.96): Gürültüyü her adımda %4 azaltıyoruz. 350 adımda gürültü neredeyse sıfıra inecektir. Bu, sistemin önce "çılgınca" arama yapmasını, sonra sakince dibe çökmesini sağlar.
- iv. Azalan Adım(lr_decay): SGD için daha yavaş (0.998), GD için daha hızlı (0.995) düşüş seçildi; çünkü SGD'nin gürültüden dolayı daha uzun süre hareket kabiliyetine ihtiyacı var.

```
# =====  
# 4. DİNAMİK BAŞLANGIÇ NOKTASI  
# =====  
if local_mins:  
    trap_x = local_mins[0][0]  
    # Sağdaki tepe noktasını bul (barrier)  
    peak_x_candidates = [x for x in all_critical_x if x > trap_x]  
    peak_x = min(peak_x_candidates) if peak_x_candidates else X_LIMIT_MAX  
    # Dinamik rastgele başlangıç  
    start_x = random.uniform(trap_x, peak_x)  
else:  
    start_x = random.uniform(X_LIMIT_MIN, X_LIMIT_MAX)  
  
print(f"\nSimülasyon Başlıyor → Dinamik Başlangıç x = {start_x:.4f}")  
  
# =====  
# 5. ÇALIŞTIRMA  
# =====  
# Batch GD  
path_batch, final_batch = smart_batch_gd(  
    start_x,  
    initial_lr=0.12,  
    lr_decay=0.995,  
    max_steps=150,  
    tolerance=1e-4  
)  
  
# Stochastic GD  
path_sgd, best_sgd = smart_sgd_annealing(  
    start_x,  
    initial_lr=0.12,  
    lr_decay=0.998,  
    initial_noise=18.0,  
    noise_decay=0.96,  
    max_steps=350  
)
```


Sistem analiz ediliyor...
-> Analitik Global Min: $x = -1.4445$
-> Dinamik Sınırlar: $[-2.86, 2.80]$

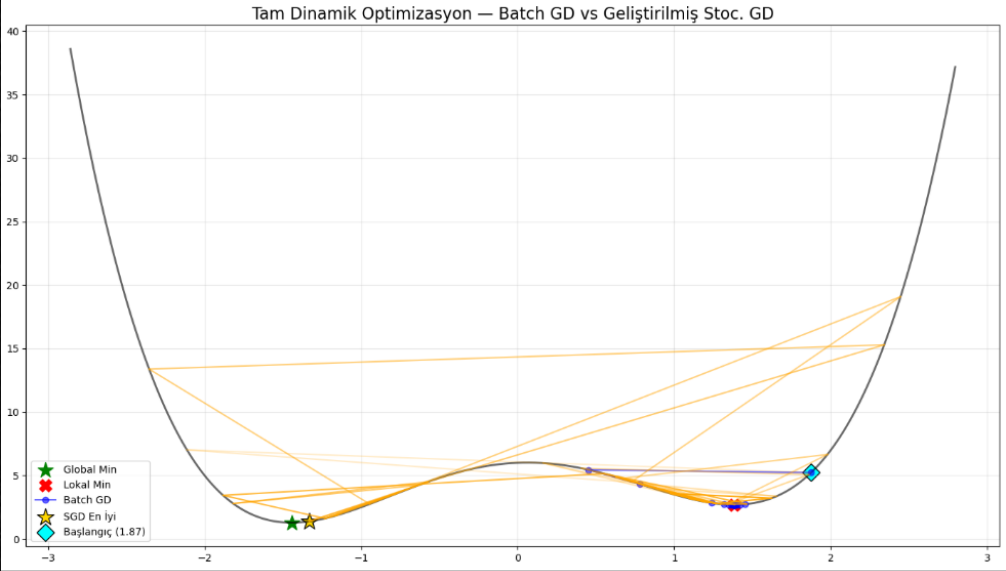
Simülasyon Başlıyor → Dinamik Başlangıç $x = 1.8032$



Batch Son: 1.3818083055099655 Loss: 2.699113564985152
SGD En iyi: -1.4519470903014169 Loss: 1.2857229537108328

Sistem analiz ediliyor...
-> Analitik Global Min: $x = -1.4445$
-> Dinamik Sınırlar: $[-2.86, 2.80]$

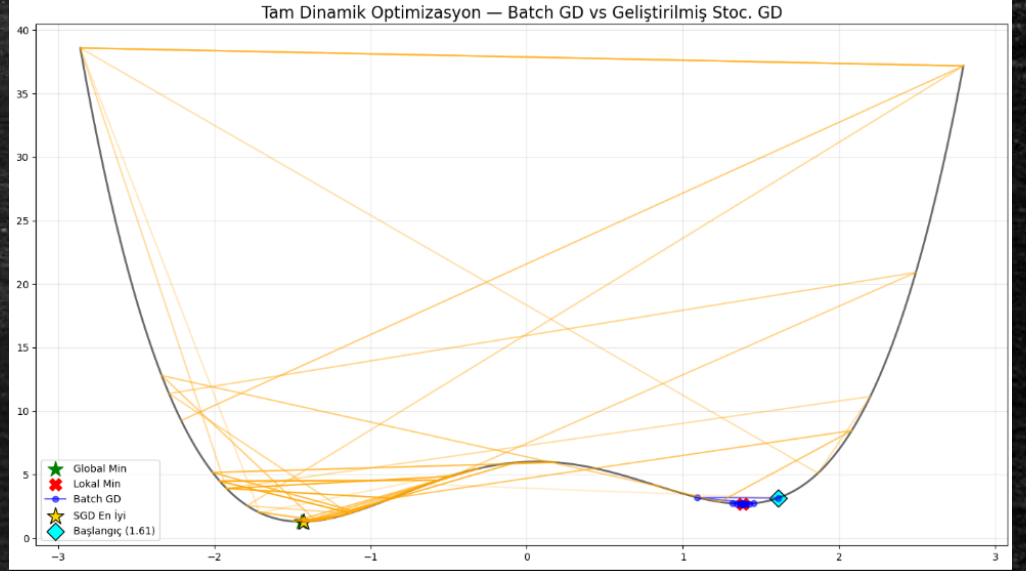
Simülasyon Başlıyor → Dinamik Başlangıç $x = 1.8744$



Batch Son: 1.3818191729738085 Loss: 2.6991135571780407
SGD En iyi: -1.3327211914133286 Loss: 1.383750118772518

Sistem analiz ediliyor...
-> Analitik Global Min: $x = -1.4445$
-> Dinamik Sınırlar: $[-2.86, 2.80]$

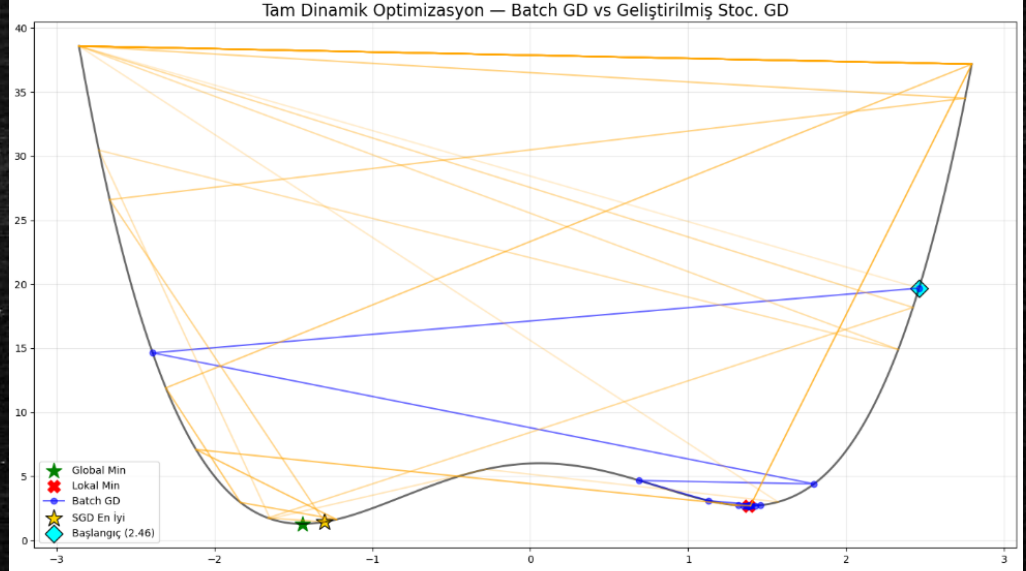
Simülasyon Başlıyor → Dinamik Başlangıç $x = 1.6103$



Batch Son: 1.3819039765431163 Loss: 2.699113556755832
SGD En iyi: -1.4288645958624169 Loss: 1.2873027605597054

Sistem analiz ediliyor...
-> Analitik Global Min: $x = -1.4445$
-> Dinamik Sınırlar: $[-2.86, 2.80]$

Simülasyon Başlıyor → Dinamik Başlangıç $x = 2.4611$



Batch Son: 1.3819208916857348 Loss: 2.699113569503023
SGD En iyi: -1.302529326439486 Loss: 1.4407974273460722

SONUÇ

Deterministik yaklaşım çok daha stabil olsa da global optimumu bulma garantisi yoktur. Stokastik yaklaşım ise başlangıçta kontrolsüz görünse de sistem soğudukça ideal çözümü çok daha yüksek ihtimalle bulur. Bu çalışma, non-convex yüzeylerde modern SGD tekniklerinin neden daha iyi sonuç verdiğini pratik olarak göstermektedir.